

Assignment - 05

Title : Binary Search Tree

Objective : To Study Some advanced data Structure
Such as Tree's.

Problem Statement :
Implement binary search tree and perform the
following operations :

- Insert (Handle insertion of duplicate entry)
- Search
- Delete
- Display tree (Traversal)
- Display - Depth of tree
- Display - Mirror image
- Create a Copy
- Display all parent node with their child nodes.
- Display leaf node's
- Display tree level wise.

Outcome : Solve problem's using algorithmic
design techniques of data Structure's.

(Note : Insertion, Deletion, Search and Traversal
are compulsory, from rest of operation, perform
Any three.)

Theory :

Binary Search tree $\Rightarrow$ Binary Search tree is a binary tree in which value's of left child is less than its parent node and value of right child is greater than its parent node.

Left Subtree $<$ root node value $<$ Right Subtree.

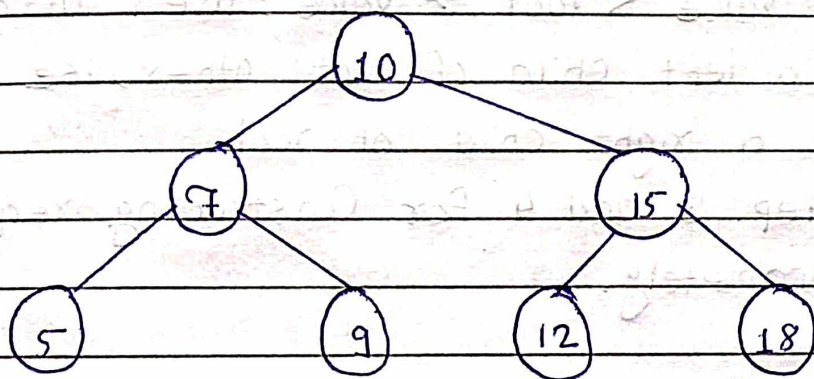


Fig. Binary tree

Operation's on Binary Search Tree :-

various operation's that can be performed on binary search tree are -

1. Insertion of a node in a binary tree.
2. Deletion of some element from the binary search tree.
3. Searching of an element in the binary tree.
4. Display of a binary tree.

Insertion of a node in binary tree:

Algorithm:

1. Read the value for the node which is to be created, and store it in a node called New.
2. Initially if ($\text{root} \neq \text{NULL}$) then $\text{root} = \text{New}$.
3. Again read the next value of node created in New.
4. If ($\text{New} \rightarrow \text{Value} < \text{root} \rightarrow \text{Value}$) then attach New node as a left child of root otherwise attach New node as a right child of root.
5. Repeat Step 3 and 4 for constructing required Binary tree Completely.

Deletion of an element from the binary tree:-

For deletion of any node from binary search tree there are three cases which are possible.

- i. Deletion of leaf node.
- ii. Deletion of a node having one child.
- iii. Deletion of a node having two children.

Algorithm:

1. Input the node's of the tree.
2. Consider the first element as the root element and insert all the element's.
3. Input the data of the node to be deleted.
4. If the node is a leaf node, delete the node directly.
5. Else if the node has one child, copy the child to the node to be deleted & delete the child.

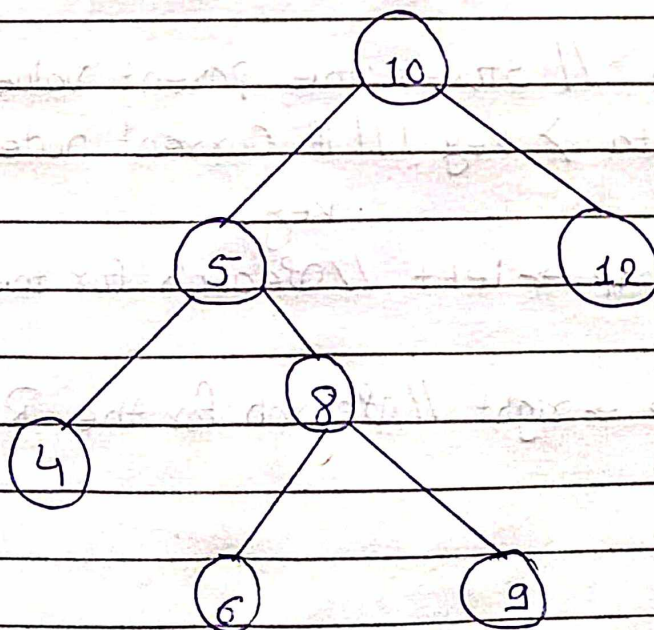
6. Else if the node has two children, find the inorder Successor of the node.

7. Copy the Content's of the inorder Successor to the node to be deleted and delete the inorder Successor.

Searching a node from binary Search tree :-

In Searching, the node which we want to search is called a key node. The key node will be compared with each node starting from root node, if the value of key node is greater than current node then we search for it on right sub-branch otherwise on left sub-branch.

If we reach to leaf node and still we do not get the value of key node then we declare "node is not present in the tree".



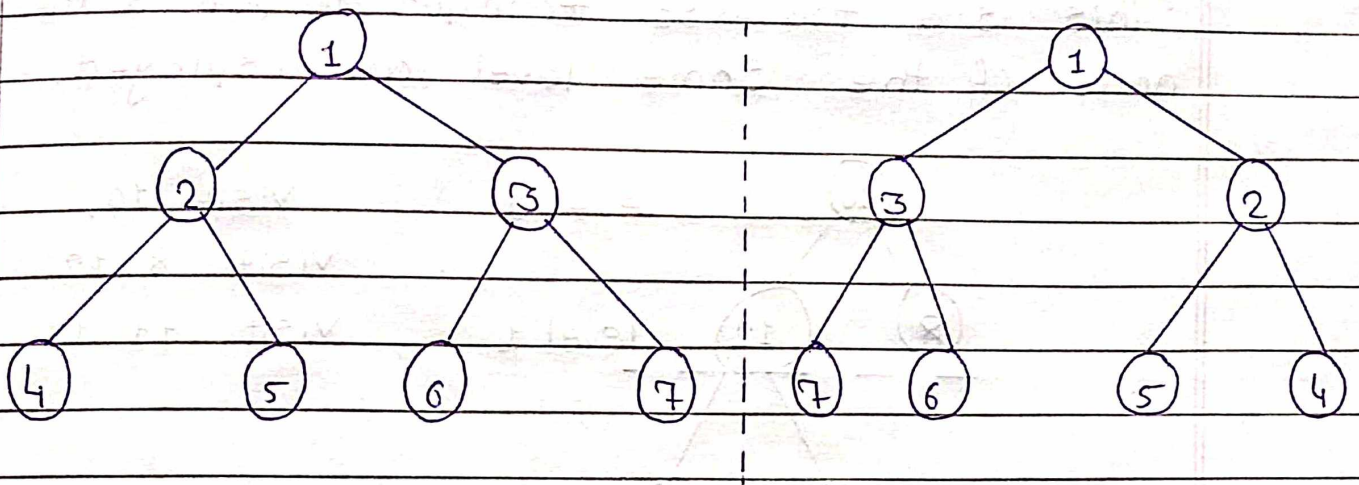
In the above tree, if we want to search for value 9. Then we will compare 9 with root node

As 9 is less than 10 we will search on left Sub-branch. Now Compare 9 with 5, But 9 is greater than 5. So we will move on right Sub-branch. Now Compare 9 with 8 but, as 9 is greater than 8 we will move on right Sub-branch. Thus the desired node can be Searched.

Pseudocode for Searching element in BST:

```
{ While (temp != NULL)
{
    if (temp → data == key)
    {
        cout << "The Element is present" << temp → data;
        break;
    }
    Parent = temp // Store's the parent value.
    if (temp → data > key) // if Current node is greater than
        key.
        temp = temp → left // Search for the left Subtree.
    else
        temp = temp → right // Search for the Right Subtree.
}
```


Mirror image of a Binary Search tree :-



Pseudocode for finding mirror image :

btree * temp :

if (root != NULL)

{

mirror (root → left);

mirror (root → right);

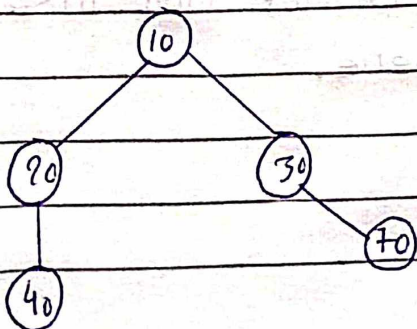
temp = root → left;

root → left = root → right;

root → right = temp;

}

Leaf's / leaf node :- Leaf node's mean's a node having no child node. AS leaf nodes are not having further links, we call leaf node external nodes.

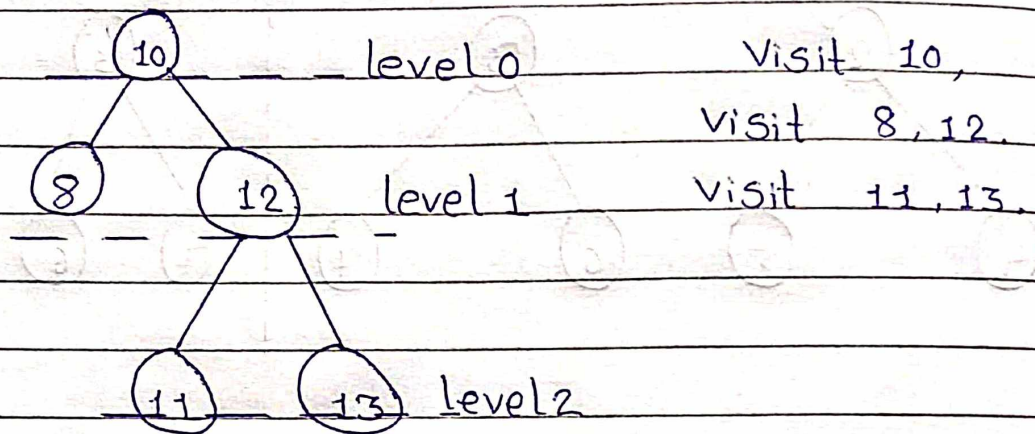


The given fig. is not of a Binary tree but,

40 & 70 are leaf's or leaf of given tree.

Level-wise Display :-

This is a traversal technique in which all the nodes of the same level are displayed.



∴ The Sequence will be 10, 8, 12, 11, 13.

In this traversal method, we start from the root node and go on printing the data in level wise manner.

An additional data structure called queue is required to perform this traversal.

Algorithm :

Step 1 : Visit the root node. Insert it in a Queue from the rear end.

Step 2 : Delete the node from front end of the queue and display it as output.

Step 3 : If its left child is not null insert the left child in the queue.

	Test Case Description	Test data	Expected Result	Actual Result	Pass / Fail
1.	Check the result when user select an option of search an element in binary tree	Search for the particular element in a binary search tree	When the element to search is found then it will print "The element is found" otherwise not	"Element is present" otherwise "Element is not present"	Pass
2.	Check the result when user select an option of insert an element in BST.	Insert particular number in BST.	Insert the element given by user in BST and if the element is already present print duplicate entry	Element given by a user get inserted in a BST.	Pass
3.	Check the result when user select an option of delete an element from BST.	Delete a particular element from BST.	Delete the element entered by user in BST.	Element Entered by the user get deleted if that is present in BST	Pass


```
#include <iostream>

#include <queue>

using namespace std;

struct Node
{
    int data;
    Node *left, *right;
};

class BST
{
    Node* root;
    Node* mirror;

public:
    BST();
    Node* getroot();
    Node* getmirror();
    void insert();
    void insert_rec(Node *);
    void del(int);
    Node* delete_Node(Node *, int);
    //void mir();
    void mirror_tree(Node *);
    void search(int);
    void display(Node *);
    void display_level();
    int height_tree(Node*);
};
```



```
BST::BST()
```

```
{
```

```
    root = NULL;
```

```
    mirror = NULL;
```

```
}
```

```
Node* BST::getroot()
```

```
{
```

```
    return root;
```

```
}
```

```
Node* BST::getmirror()
```

```
{
```

```
    return mirror;
```

```
}
```

```
void BST::insert()
```

```
{
```

```
    Node* temp, *newNode;
```

```
    char ch = 'y';
```

```
    while(ch == 'y' || ch == 'Y')
```

```
    {
```

```
        newNode = new Node();
```

```
        cout<<"\nEnter data for new Node: ";
```

```
        cin>>newNode->data;
```

```
        newNode->left = newNode->right = NULL;
```

```
        if(root == NULL)
```

```
            root = newNode;
```

```
        else
```



```

{
    temp = root;
    check:
    if(newNode->data < temp->data)
    {
        if(temp->left != NULL)
        {
            temp = temp->left;
            goto check;
        }
        else
        {
            temp->left = newNode;
        }
    }
    else if(newNode->data > temp->data)
    {
        if(temp->right != NULL)
        {
            temp = temp->right;
            goto check;
        }
        else
        {
            temp->right = newNode;
        }
    }
}

cout<<"\nEnter y or Y to add more Nodes: ";
cin>>ch;
}

```

```
}
```

```
void BST::del(int x)
```

```
{
```

```
    root = delete_Node(root, x);
```

```
}
```

```
Node* BST::delete_Node(Node *t, int x)
```

```
{
```

```
    Node *temp;
```

```
    if(t == NULL)
```

```
    {
```

```
        cout<<"\nElement not found!";
```

```
        return t;
```

```
    }
```

```
    if(x < t->data)
```

```
    {
```

```
        t->left = delete_Node(t->left, x);
```

```
        return t;
```

```
    }
```

```
    else if(x > t->data)
```

```
    {
```

```
        t->right = delete_Node(t->right, x);
```

```
        return t;
```

```
    }
```

```
    else
```

```
    {
```

```
        if(t->left == NULL && t->right == NULL)
```

```
        {
```



```

        temp = t;
        delete temp;
        return NULL;
    }
    if(t->left == NULL)
    {
        temp = t;
        t = t->right;
        delete temp;
        return t;
    }
    if(t->right == NULL)
    {
        temp = t;
        t = t->left;
        delete temp;
        return t;
    }
    temp = t->right;
    while(temp->left != NULL)
        temp = temp->left;
    t->data = temp->data;
    t->right = delete_Node(t->right, temp->data);
    return t;
}

return NULL;
}

```

```

void BST::mirror_tree(Node* t)
{
    Node* temp;

```

```

if(t!=NULL)
{
    temp = t->left;
    t->left = t->right;
    t->right = temp;

    mirror_tree(t->left);
    mirror_tree(t->right);
}
else
    return;
}

```

```

void BST::search(int no)

```

```

{

    Node *temp;
    temp = root;

    check:
    if(no == temp->data)
    {
        cout<<"\nData found!";
        return;
    }
    if(no < temp->data)
    {
        if(temp->left != NULL)
        {
            temp = temp->left;

```



```

        goto check;
    }
    else
    {
        cout<<"\nData doesnt exist.";
        return;
    }
}
else if(no > temp->data)
{
    if(temp->right != NULL)
    {
        temp = temp->right;
        goto check;
    }
    else
    {
        cout<<"\nData doesnt exist.";
        return;
    }
}
}

```

```

void BST::display(Node *root)
{
    if(root == NULL)
        return;
    else
    {
        display(root->left);
        cout<<" "<<root->data<<" ";
    }
}

```

```

        display(root->right);
    }
}

void BST::display_level()
{
    queue<Node*> q, q2;
    Node* curr;
    Node* temp = root;
    if(temp == NULL)
        return;

    q.push(temp);
    q.push(NULL);

    while (q.size() > 1)
    {
        curr = q.front();
        q.pop();

        if (curr == NULL)
        {
            q.push(NULL);
            cout << "\n";
        }

        else
        {
            if(curr->left)
                q.push(curr->left);

```



```

        if(curr->right)
            q.push(curr->right);

        cout << curr->data << " ";

    }

}

```

```

int BST::height_tree(Node *t)
{
    int hlt, hrt;
    if(t == NULL)
        return 0;
    else
    {
        hlt = height_tree(t->left) + 1;
        hrt = height_tree(t->right) + 1;
    }
    if(hlt > hrt)
        return hlt;
    return hrt;
}

```

```

int main()
{
    BST tree;
    int x, choice;
    char ch = 'y';

    while(1)

```

```

{

    cout<<"\n\nBinary Search Tree Operations ";

    cout<<"\n1. Insert.";
    cout<<"\n2. Delete.";
    cout<<"\n3. Search.";
    cout<<"\n4. Mirror Image.";
    cout<<"\n5. Display.";
    cout<<"\n6. Display Level-wise.";
    cout<<"\n7. Height of the tree.";
    cout<<"\n8. Exit.";
    cout<<"\nEnter your choice: ";
    cin>>choice;

    switch(choice)
    {
        case 1:
            tree.insert();
            break;
        case 2:
            cout<<"\nEnter data to delete: ";
            cin>>x;
            tree.del(x);
            break;
        case 3:
            cout<<"\nEnter number to search: ";
            cin>>x;
            tree.search(x);
            break;
        case 4:
            cout<<"\nMirror Image of tree: ";
            tree.mirror_tree(tree.getroot());
    }
}

```

```

        tree.display(tree.getroot());
        tree.mirror_tree(tree.getroot());
        break;
    case 5:
        cout<<"\nInorder traversal of tree: ";
        tree.display(tree.getroot());
        break;
    case 6:
        cout<<"\nLevel-Order traversal of tree: ";
        tree.display_level();
        break;
    case 7:
        cout<<"\nHeight of the tree: "<<tree.height_tree(tree.getroot());
        break;
    case 8:
        return 0;
    }
}
return 0;
}

```